

NBA Statline Forecasting

Context-Aware Machine Learning for Player Performance Prediction

DSCI 4350: Data Science Capstone

Spring 2026

Team Members:

Arul Selvakumar

1. Introduction

1.1 Motivation

Accurate forecasting of NBA player performance has proved to be a problem across numerous real-world domains, namely betting companies, sports books, and analytics teams. Player statistical output is determined by a variety of complex factors including season averages, usage rate, roster availability, home-court advantage, team pace and defensive dynamics, just to name a few. Traditional forecasting methods rely on season averages to capture these game-specific concepts resulting in weak predictions that lack true meaningful analysis. This project focuses on the challenge of constructing a predictive ML system that accounts for pregame contextual variables to create more precise performance forecasts for NBA players.

1.2 Task Specification

The objective of this project is to continuously predict three target variables: points (PTS), rebounds (REB), assists (AST). Predictions are generated using only pregame information available before the start of a selected game, maintaining and simulating realistic deployment measures. The entire project is framed as a supervised regression problem.

1.3 Problem Formulation

Pregame features are found in player-level and team-level data.

Inputs: season averages, rolling statistical averages (last 5, last 10 games), usage rates, roster availability signals, pace indicators, opponent defensive metrics.

Outputs: predictions for PTS, REB, AST for each player-game instance.

Constraints: postgame features removed during preprocessing, time-based training on a 2-year split used to make sure no future data entry contaminates the training process.

2. Data Acquisition and Preparation

2.1 Data Collection Procedures

Dataset collected by accessing official NBA statistics using established `nba_api` Python package. Data spans two entire seasons (2023-24, 2024-25) at player granularity – each row represents an individual player’s performance in a specific game. Temporal split on data with the 2023-2024 season serving as the training set (80/20 training split) and the 2024-2025 season serving as the held-out testing set. Using this time-based split prevents data leakage and simulates correct deployment conditions.

Table 1: Dataset Summary

Attribute	Value
Player-Game Rows	~52,707
Players	694
Games	~2,460
Training Season	2023–24
Testing Season	2024–25
Source	<code>nba_api</code> (Python)

2.2 Licensing and Usage Rights

The data is publicly available through the NBA’s official statistics API and is used strictly for educational purposes within the scope of this capstone project. No redistribution or commercial use of the data is intended.

2.3 Ethical Considerations

The dataset consists exclusively of publicly available sports statistics and contains no personal, sensitive, or private information. No ethical concerns related to privacy or fairness arise from the use of this data.

2.4 Limitations

Data collection process showed several limitations. The dataset does not incorporate real-time reporting on injury status that may affect player availability after initial lineup is announced. Potential bias may arise from incomplete availability data. Strong contextual variables such as play-calling tendencies or in-game rotations are not captured. Dataset is currently limited to two seasons as well, which does not fully capture long-term trajectory.

2.5 Preprocessing

All records were sorted by player and game date to keep temporal ordering consistent – critical for interpretation of rolling features. Missing values handled using forward fill for rolling averages as well as sensible defaults applied where no prior data existed. Only fully constructed player-game row logs are available for use by the model. All postgame features removed from the dataset to prevent data leakage with only pregame features retained.

2.6 Feature Engineering

A systematic feature versioning approach was used to continuously build richer feature representations. Each feature version focuses on adding new contextual information.

Table 2: Feature Version Descriptions

Version	Description
Baseline	Season-long averages for PTS, REB, and AST. Serves as the simplest prediction baseline.
V0	Adds simple opponent context (defensive rating, pace). Adds home/away indicator.
V1	Incorporates availability signals . Captures the missing production from injured or resting teammates.
V2	Introduces usage rate and matchup context . Reflecting the player’s expected role in each game.
V3	Advanced interaction features: usage × minutes, pace-adjusted statistics, and trend features comparing last-5-game to last-10-game rolling averages (L5 v L10).

V3 feature set displays the most expressive version of modeling in efforts to capture nonlinear contextual relationships that drive NBA player performance. These advanced interaction features code for information that simple interaction features cannot represent (i.e., how a player's output changes when playing at a faster pace or against a weaker defense).

2.7 Cleaning

Data cleaning procedures used in this process include the removal of duplicate records, the standardization of feature formats across seasons, and most importantly the filtering of inconsistent entries where identifiers did not align. This ensured the creation of a reliable dataset for model training and evaluation.

3. Data Science Methodology

3.1 Models Used

Three regression models were selected for use to demonstrate progression from simpler to more complex algorithms – goal being higher performance.

Linear Regression: Standard ordinary squares model used as the interpretability baseline. LR provides a reference point for understanding relationships between features and target variables. Predictions can be fully represented in terms of feature coefficients.

Random Forest Regressor: Ensemble method that averages predictions of numerous decision trees. RF captures nonlinear relationships and feature interactions without the need for explicit feature engineering. Provides crucial feature importance estimates.

XGBoost Regressor: Gradient-boosted tree ensemble designed to predict performance on structured tabular data. Trees are constructed sequentially in XGB with each tree correcting errors of the previous. Regularization included for optimization. Expected to be the strongest predictive model.

3.2 Model Justification

The regression task requires models capable of predicting continuous outputs. Linear regression establishes a simple baseline, allowing performance gains from more complex models to be measured clearly. Tree-based models (RF + XGB) are suited for structured tabular data and handle nonlinear feature interactions. XGBoost has consistently demonstrated state-of-the-art performance on tabular regression benchmarks and is the expected best performer for this task.

3.3 Model Construction

The modeling pipeline evolved across multiple stages. Versions V0 through V2 introduced incremental feature improvements, each adding new categories of pregame context. Version V3 introduced advanced interaction and trend features that capture nonlinear relationships in the data. Version V4 applied hyperparameter tuning to the V3 feature set using grid search, optimizing parameters such as learning rate, maximum tree depth, number of estimators, and regularization strength. The combination of V3's enriched feature set with V4's optimized hyperparameters produced the best-performing models.

4. Evaluation and Experiments

4.1 Purpose of Evaluation

The purpose of this evaluation is to measure how systematic feature engineering and hyperparameter tuning improve prediction accuracy on unseen future data. By training on the 2023-24 season and testing on the 2024-25 season, the evaluation simulates a realistic deployment scenario where the model must generalize to new games, new opponents, and evolving player roles.

4.2 Experimental Setup

Experimental setup uses a strict time-based holdout split. The training set consists of all player-game records from the 2023-24 NBA regular season, and the test set consists of all records from the 2024-25 season. No random sampling or cross-validation is used. Design choice reflects real-world conditions where predictions must be made on future games. All models were implemented using scikit-learn and XGBoost libraries.

4.3 Evaluation Metrics

Mean Absolute Error (MAE) used as the primary evaluation metric. MAE measures the average magnitude of prediction errors in the same units as the target variable. MAE is robust to outliers and provides an intuitive measure of model accuracy.

Root Mean Squared Error (RMSE) is used as a secondary evaluation metric. RMSE penalizes larger errors more heavily than MAE due to its squaring operation, making it useful for identifying models that produce occasional large prediction errors. MAE and RMSE together provide a comprehensive view of model performance.

4.4 Results

The best-performing model across all three target variables is XGBoost with V4 hyperparameter tuning, as summarized in Table 3 and shown in Figure 1.

Table 3: Best Model Performance (XGBoost, V4 results)

Target	MAE	RMSE
Points (PTS)	2.195	3.012
Rebounds (REB)	0.908	1.258

Assists (AST)	0.602	0.887
---------------	-------	-------

```
Best models by target:
```

	run_id	version	target	model	tuned	mae	rmse	feature_count	runtime_seco
0	V4_target_ast_XGBRegressor	V4	target_ast	XGBRegressor	True	0.602005	0.886846	78	3.015
1	V4_target_reb_XGBRegressor	V4	target_reb	XGBRegressor	True	0.908092	1.257660	78	3.174
2	V4_target_pts_XGBRegressor	V4	target_pts	XGBRegressor	True	2.195287	3.012371	78	2.848

Figure 1: Best models by target variable, showing XGBoost, V4 as top performer across all three targets.

Table 4 and Figure 2 below present a comparison of all model-target combinations, illustrating the performance hierarchy across Linear Regression, Random Forest, and XGBoost for each target variable. Shows what enabled us to select XGBoost as the best evaluation model.

Table 4: Complete Model Comparison (V4)

Target	Model	MAE	RMSE
AST	XGBoost (Tuned)	0.602	0.887
AST	Random Forest (Tuned)	0.680	1.002
AST	Linear Regression	0.846	1.193
PTS	XGBoost (Tuned)	2.195	3.012
PTS	Random Forest (Tuned)	2.458	3.388
PTS	Linear Regression	2.947	3.947
REB	XGBoost (Tuned)	0.908	1.258
REB	Random Forest (Tuned)	1.015	1.404
REB	Linear Regression	1.251	1.692

	run_id	version	target	model	tuned	mae	rmse
8	V4_target_ast_XGBRegressor	V4	target_ast	XGBRegressor	True	0.602005	0.886846
7	V4_target_ast_RandomForestRegressor	V4	target_ast	RandomForestRegressor	True	0.680376	1.002479
6	V4_target_ast_LinearRegression	V4	target_ast	LinearRegression	False	0.845995	1.193216
2	V4_target_pts_XGBRegressor	V4	target_pts	XGBRegressor	True	2.195287	3.012371
1	V4_target_pts_RandomForestRegressor	V4	target_pts	RandomForestRegressor	True	2.457605	3.387816
0	V4_target_pts_LinearRegression	V4	target_pts	LinearRegression	False	2.946712	3.947072
5	V4_target_reb_XGBRegressor	V4	target_reb	XGBRegressor	True	0.908092	1.257660
4	V4_target_reb_RandomForestRegressor	V4	target_reb	RandomForestRegressor	True	1.014672	1.404213
3	V4_target_reb_LinearRegression	V4	target_reb	LinearRegression	False	1.250617	1.691776

Figure 2: Tuning results across all model-target combinations.

4.5 Visualizations

Figures 3 through 5 present the MAE progression from the baseline feature set through V4 for each target variable. These charts illustrate the impact of each feature engineering stage and hyperparameter tuning on overall prediction accuracy.

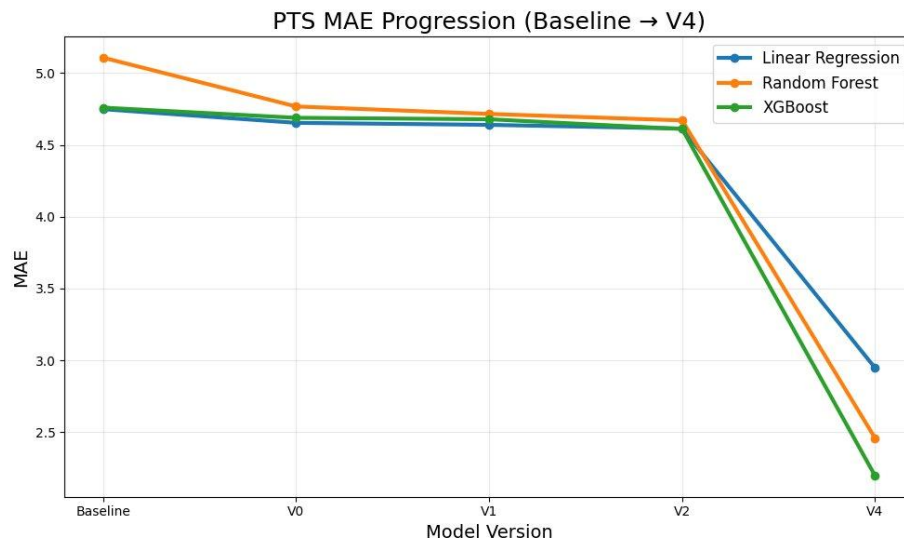


Figure 3: PTS MAE progression from Baseline through V4.

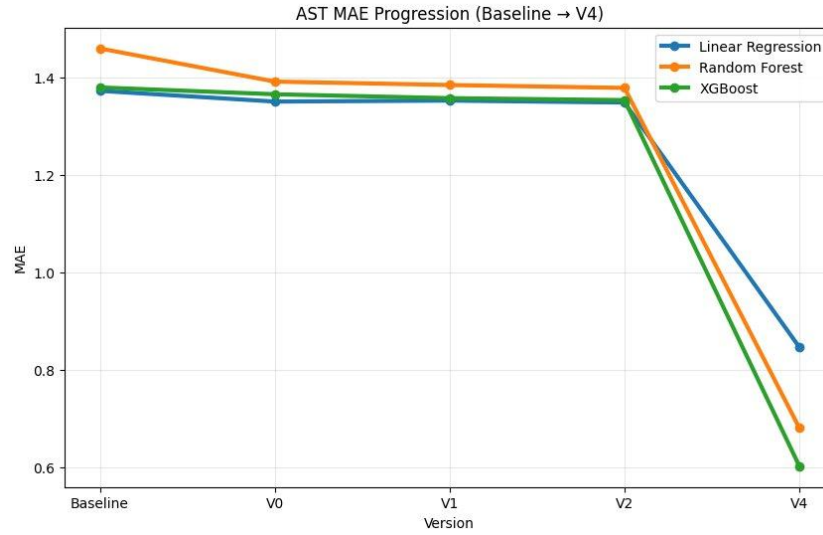


Figure 4: AST MAE progression from Baseline through V4.

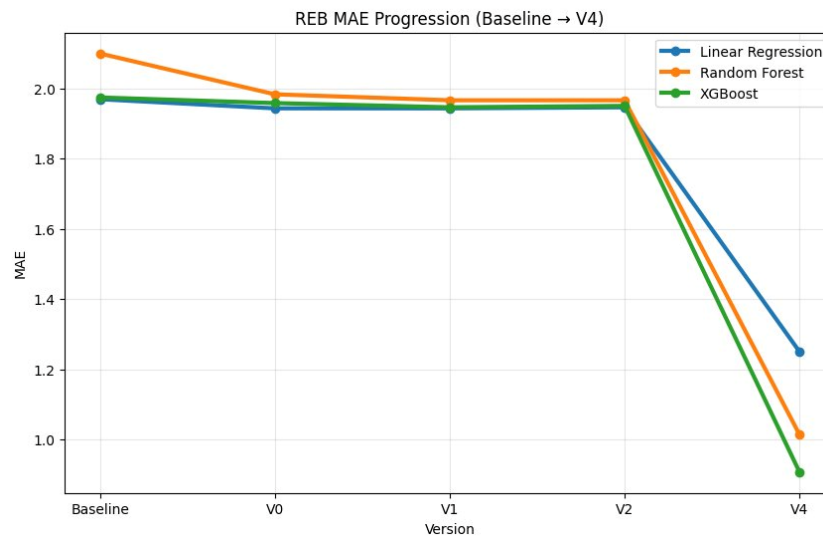


Figure 5: REB MAE progression from Baseline through V4.

4.6 Data Analysis

Performance improves gradually from the baseline through V2, showing that each incremental addition of contextual features contributes to prediction accuracy. The most substantial performance improvement occurs between V2 and V4, driven by the introduction of V3's interaction and trend features combined with V4's hyperparameter tuning. XGBoost consistently outperforms both Random Forest and Linear Regression across all three target variables, confirming its suitability for this structured regression task. The performance hierarchy (XGBoost > Random Forest > Linear Regression) is preserved across all targets, suggesting that the

nonlinear patterns captured by tree-based models are consistently present in the data. The combination of V3 contextual features and V4 tuning produces the largest cumulative gains, validating the feature engineering strategy as the primary driver of model improvement.

5. Deployment

5.1 Deployment Strategy

The final trained XGBoost (V4) models are deployed as a Streamlit web application. Streamlit was specifically selected for its simplicity in building interactive data applications with Python and its ability to render model predictions alongside user inputs in real time. The application simulates a real-world pregame inference pipeline, accepting pregame contextual features and generating predicted statlines.

5.2 Application Features

Users can select a random held-out game from the 2024–25 test set and the application displays the pregame context for that game. The user is then prompted to input their own predictions for PTS, REB, and AST. The system generates the model's predictions using the trained V4 XGBoost models, retrieves the actual game results, and compares all three sets of values. An error calculation determines the total absolute error for both the user and the model, and a winner is declared based on which set of predictions was closer to the actual outcome. This interactive “Can You Beat the Model?” feature engages users with the prediction task and demonstrates the model's practical utility.

NBA Statline Forecasting Demo

Historical replay demo using held-out 2024–25 games.

Generate New Held-Out Game

Figure 6: Streamlit App UI

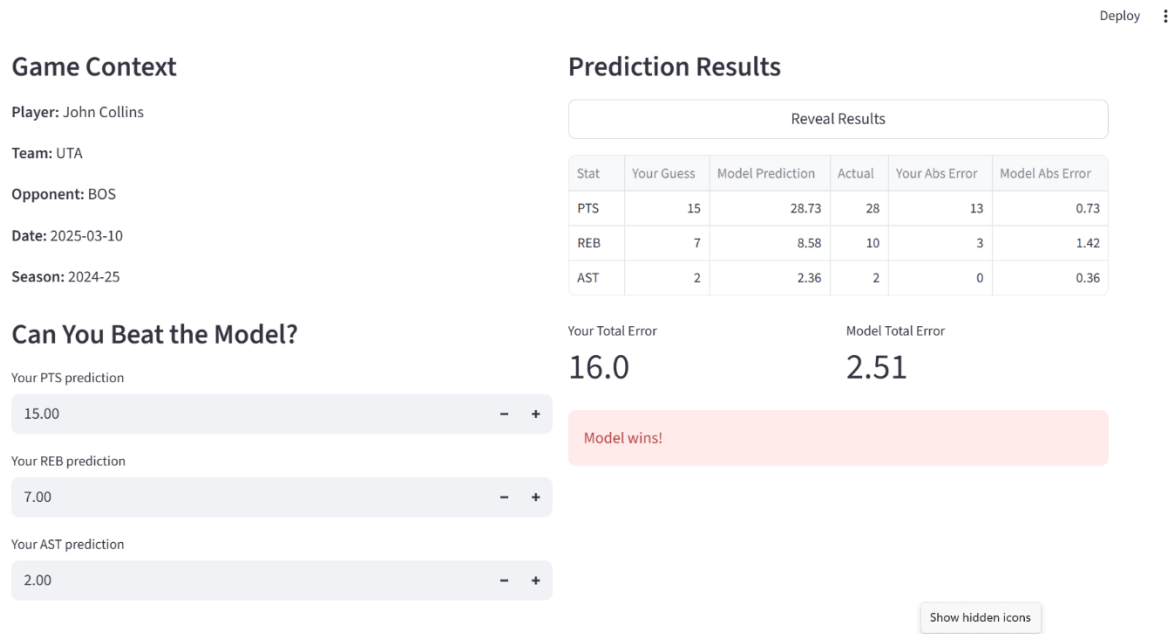


Figure 7: Interactive Demo (User vs Model)

5.3 Deployment Requirements

The application requires a Python environment with the following libraries: streamlit, pandas, numpy, scikit-learn, xgboost, nba_api, and joblib for model serialization. Application runs locally via the Streamlit command-line interface.

5.4 Scalability

The current deployment is designed for local execution and demonstration purposes. For production scalability, the application could be extended to an API-based deployment using frameworks enabling integration with external systems. Real-time data pipelines could be incorporated to pull live pregame features from the NBA API, allowing the model to generate predictions for upcoming games automatically.

6. Conclusion and Summary

6.1 Summary of Work

This project developed an end-to-end machine learning pipeline for NBA statline forecasting. Pipeline encompasses data collection from the NBA API (nba_api), systematic feature engineering across five versioned feature sets, model training and evaluation using three regression algorithms, and deployment as an interactive Streamlit web application. The project work demonstrates that context-aware pregame features, combined with optimized gradient-boosted models, can produce accurate and practical player performance predictions.

6.2 Key Findings

The most important finding is that context-aware features significantly improve prediction accuracy over simple season averages. The V3 interaction features – combining usage, pace, and trend information – provided the largest single improvement in model performance. XGBoost consistently achieved the best results across all three target variables, with final MAE scores of approximately 2.19 for Points, 0.91 for Rebounds, and 0.60 for Assists. The combination of V3 feature engineering and V4 hyperparameter tuning produced the largest cumulative performance gains.

6.3 Issues and Shortcomings

The model currently does not incorporate real-time data, otherwise meaning that it cannot adapt to late-breaking lineup changes or injury reports. Potential feature bias may exist due to incomplete availability data or inconsistent reporting across teams. No deep learning models were explored in this project, which may limit the ability to capture more complex temporal patterns in player performance data.

6.4 Future Work

Neural network architectures, such as recurrent neural networks or transformers, could be explored to capture sequential dependencies in player performance over time. Real-time data integration would allow the model to incorporate up-to-the-minute injury and lineup information. An API-based deployment with a more robust dashboard interface would improve accessibility and scalability. Enhanced lineup modeling that accounts for the full five-player lineup context and opponent lineup matchups could further improve prediction accuracy.